

ClassLedger — Frontend Implementation

Brief

For building in Cursor · Next.js + TypeScript · Frontend-only phase (mock data), designed to plug into a real backend later without rework.

1. What we're building

ClassLedger is a student and teacher management tool for a small coaching class — attendance, marks, fees, and chapter/topic-level performance, with a head-teacher dashboard that surfaces which students need attention. This phase builds **only the frontend**: every screen, fully clickable, running on realistic mock data. No backend, no database, no real auth yet. The goal is a UI that looks and behaves like the final product, structured so a backend can be wired in later by swapping a data layer — not by rewriting components.

Guiding rule for this phase

- Build every screen as if data were real — compute red flags, trends, and fee status from the mock data using the actual business rules (section 6), not hardcoded labels.
- Never let a component fetch or shape data itself. Components receive typed props; a separate data layer (section 8) decides where that data comes from.
- Keep student-only views structurally separate from teacher views from day one, even though there's no real auth yet — this is a privacy requirement, not a nice-to-have.

2. Tech stack & setup

Layer	Choice	Notes
Framework	Next.js 14+ (App Router)	Use the App Router (app/ directory), not Pages Router.
Language	TypeScript	Strict mode on. Every mock data file is typed (section 8).
Styling	Tailwind CSS	Utility classes; see design tokens in section 4.
Icons	lucide-react	Lightweight, matches the prototype's icon style.
Charts (later)	recharts	Only needed for trend lines / batch comparisons.
State	React state + Context	No external state library needed for frontend-only phase.

Setup command

```
npx create-next-app@latest classledger --typescript --tailwind --app
cd classledger
npm install lucide-react
```

3. Folder structure

```
app/
(auth)/login/page.tsx
(auth)/signup/page.tsx
(auth)/signup/success/page.tsx
(teacher)/dashboard/page.tsx -> Overview
(teacher)/dashboard/students/page.tsx -> Students list
(teacher)/dashboard/students/[id]/page.tsx -> Student profile
(teacher)/dashboard/tests/page.tsx -> Tests list
(teacher)/dashboard/tests/create/page.tsx -> Create test
(teacher)/dashboard/tests/[id]/page.tsx -> Test result view
(teacher)/dashboard/fees/page.tsx -> Fees
```

```

(teacher)/dashboard/attendance/page.tsx -> Mark attendance
(student)/me/page.tsx -> Student's own view
layout.tsx, globals.css
components/
layout/Navbar.tsx, Tabs.tsx
ui/Card.tsx, Tag.tsx, StatTile.tsx, DataTable.tsx, ProgressBar.tsx, Button.tsx
dashboard/AttendanceRing.tsx, FlagList.tsx, NoticeBoard.tsx
students/StudentTable.tsx, SubjectCard.tsx, ChapterTopicList.tsx
tests/TestTable.tsx, AIQuestionGenerator.tsx
lib/
types.ts -> all shared TypeScript interfaces (section 8)
data/ -> mock data + data-access functions (section 9)
rules.ts -> business rule functions (section 6)

```

4. Design tokens

Base theme is white/light. Reuse these as Tailwind custom colors (tailwind.config.ts) so every screen stays visually consistent with the approved prototype.

Token	Hex	Used for
bg / surface	#ffffff	Page and card background
surface-muted	#f6f6f4	Hero/stat blocks, table stripes
border	#e6e4dd	All hairline borders
text	#1a1a18	Primary text
text-muted	#6b6a63	Secondary/caption text
accent	#185fa5	Primary buttons, links, active tab, AI icon
accent-bg	#e6f1fb	Accent tag backgrounds
danger / danger-bg	#a32d2d / #fcebeb	Red flags, overdue fees
success / success-bg	#3b6d11 / #eaf3de	Paid, improving trend
warn / warn-bg	#9a6b12 / #fbf1de	Partial fees, average level

Radius: 10–12px on cards/buttons, pill (20px) on tags. Font: system UI stack (-apple-system, Segoe UI, Roboto). Base text 13–14px, headings 15–22px semibold.

5. Screens to build

These map directly to the approved clickable prototype. Build in this order — each is independently useful and testable.

#	Screen	Route	Key elements
1	Login	/login	Institute / Student toggle tabs, email+password or roll+password, validation
2	Institute signup	/signup	3-step form: institute details → head teacher account → review
3	Signup success	/signup/success	Confirmation tick, 'we will contact you' message, continue button
4	Overview	/dashboard	Greeting, attendance ring, stat tiles, batch cards, collapsible flagged-students, notice board, quick actions, AI icon
5	Students list	/dashboard/students	Search by batch + name, table (Today/Attendance/Trend/Fees/Flag), row → profile
6	Student profile	/dashboard/students/[id]	Header + call/report actions, tiles, flag reason, subject cards, chapter/topic breakdown, fees summary
7	Tests list	/dashboard/tests	Table (subject/chapter/date/batch/status/avg), create test button
8	Create test	/dashboard/tests/create	Test detail form + AI question generator panel (generate, edit, remove, add manually)
9	Fees	/dashboard/fees	Tiles (collected/pending/overdue), student table, remind / mark paid
10	Mark attendance	/dashboard/attendance	Per-student present/late/absent for one session
11	Add student	/dashboard/students/new	Name, batch, parent phone, parent email
12	Student's own view	/me	Own attendance/average, subject cards, chapter/topic analysis, read-only notices — never another student's data

6. Business rules to encode now

Even on mock data, compute these — don't hardcode the resulting label. This is what makes the frontend trustworthy to demo and trivial to re-point at real data later.

Rule	Logic
Red flag	Same subject's score dropped in 3 consecutive tests, OR attendance < 80%. Thresholds should be constants in lib/rules.ts, not magic numbers in components.
Trend	Latest overall average vs previous test: +2 or more = Improving, -2 or more = Dropping, else Steady.
Fee status	Paid = balance 0. Overdue = due date passed and balance > 0. Partial = balance > 0, due date not yet passed.
Pass mark	40% — used to count 'below pass mark' in test results.
Topic level	Good ≥75%, Average 50–74%, Needs improvement <50%. Show 'not enough data' below ~5 attempted questions in a topic.

7. Privacy rule — build the boundary now

A student must never see another student's name or marks — only batch averages. Even without real auth yet, structure routes so teacher-only screens live under a (teacher) route group and the student view under a separate (student) group, and have student-facing components accept only that one student's data as props —

never the full class list. This makes it structurally impossible to leak classmate data once real auth is added, instead of relying on remembering to filter it later.

8. Shared TypeScript types (lib/types.ts)

```
export interface Batch { id: string; name: string; studentCount: number }

export interface Student {
  id: string; name: string; batchId: string; parentPhone: string; parentEmail?: string;
  attendancePct: number; overallAverage: number; feeStatus: 'Paid' | 'Partial' | 'Overdue';
  flagged: boolean; flagReason?: string;
}

export interface SubjectScore {
  subject: 'Maths' | 'Chemistry' | 'Physics'; latestScore: number;
  last4Scores: number[]; batchAverageDiff: number; subjectAttendancePct: number;
}

export interface Topic { id: string; name: string; chapterId: string; scorePct: number;
  attempted: number }
export interface Chapter { id: string; name: string; subject: string; scorePct: number; topics:
  Topic[] }

export interface Question {
  id: string; text: string; type: 'MCQ' | 'Short answer' | 'Numerical';
  marks: number; chapterId: string; topicId: string; answer?: string;
}

export interface Test {
  id: string; subject: string; chapterId: string; batchId: string | 'All';
  date: string; maxMarks: number; status: 'Upcoming' | 'Marks pending' | 'Done';
  batchAverage?: number; questions: Question[];
}

export interface FeeRecord {
  studentId: string; paid: number; balance: number; dueDate: string;
  status: 'Paid' | 'Partial' | 'Overdue';
}

export interface Notice { id: string; text: string; postedAt: string; batchId: string | 'All' }
```

9. Mock data layer — the piece that makes this scalable

The single most important pattern in this phase: components never import mock arrays directly. They call a function from lib/data/. Today that function returns mock data; later it becomes a real fetch call. The component doesn't change.

```
// lib/data/students.ts
import { Student } from '../types'

const MOCK_STUDENTS: Student[] = [ /* ...typed mock rows... */ ]

export async function getStudents(batchId?: string): Promise {
  // PHASE 1 (now): return mock data
  return batchId ? MOCK_STUDENTS.filter(s => s.batchId === batchId) : MOCK_STUDENTS

  // PHASE 2 (later): replace the line above with –
  // const res = await fetch(`/api/students?batchId=${batchId ?? ''}`)
  // return res.json()
}

export async function getStudentById(id: string): Promise {
```

```
return MOCK_STUDENTS.find(s => s.id === id) ?? null
}
```

Apply the same pattern to `getBatches()`, `getTests()`, `getFees()`, `getNotices()`. Pages call these functions (as async server components, or via a simple hook) — never import the mock arrays directly.

10. Build order & what changes later

- **Now — Frontend, mock data:** build all 12 screens from section 5 in order, using `lib/data/*` functions and `lib/rules.ts` for every computed value.
- **Next — Wire to backend:** build real API routes (or a separate backend), then edit only the inside of each `lib/data/*.ts` function to call that API. Components, types, and pages stay untouched.
- **Then — Real auth:** add real login/session and use it to gate the (teacher) and (student) route groups that were already separated in section 7.

11. Reference

Clickable prototype (approved design reference, build pixel/behaviour-close to this):

<https://claude.ai/artifact/8LG94MQcDX1jQFakFFdNsr>